EECE 7205: Fundamentals of Computer Engineering In-Class Assignment #01

Katherine Tse
Email: tse.kat@northeastern.edu
NUID: 003936627

Due: September 29, 2026

1 Problem 1: Bubble Sort

1.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Bubble Sort algorithm.
- **Input:** an integer array and the size of said array n.
- **Output:** during the algorithm, print every pass; after the algorithm, print the sorted array and the total number of comparisons and swaps
- **Constraints:** no built-in sort used; two functions - `printArray` and `bubbleSort`; all array printing utilizing the corresponding functions; sort occurs in place
- **Edge Cases:** pre-sorted array (best case), reverse sorted array (worst case), duplicate values, array of size 0 or 1

1.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `bubbleSort(int arr[], int n)`	Section 4, 8
2	Call `printArray` every pass	See outer loop of `bubbleSort` for implementation, see Section 9 for output
3	Use a `bool swapped` flag to end function early	See code and Run 2 output
4	Count comparisons and swaps and print them	See code and output in Section 9
5	Run a sorted copy of A, report passes and comparisons, and explain	See Section 9 for output and Section 10 for explanation

1.3 Solution Approach

Bubble sort traverses the array from left to right and compares each neighbor. If value on the left is larger, it swaps the two. Going from the start to the end of the array one time is a single pass. Each pass, the largest element in the array as far right as possible to the end of the unsorted section of the array. After k passes, k values from the end are sorted.
Design Decisions:

- Each pass only needs to look at the unsorted portion of the array (n-2-i) [ex. pass 1 -> 7 comps, pass 2 -> 6 comps, etc.]
- The `swapped` flag is reset for every pass and tracks if a swap was made during it. If a swap was not made then the neighbors are in order and the array is sorted. The function can exit.

1.4 Pseudocode

Algorithm 1.1 Bubble Sort

```
1: procedure PRINTARRAY(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[k] followed by a space
4:     end for
5:     PRINT newline
6: end procedure
7: procedure BUBBLESORT(arr, n, comps, swaps)
8:     PRINT "Original: " ; CALL printArray(arr, n)
9:     for pass ← 0 TO n − 2 do
10:        swapped ← FALSE
11:        for j ← 0 TO  do
12:            if arr[j] > arr[j + 1] then
13:                SWAP(arr[j], arr[j + 1])
14:                swapped ← TRUE
15:                swaps ← swaps + 1
16:            end if
17:            comps ← comps + 1
18:        end for
19:        PRINT "Pass ", pass, ": " ; CALL printArray(arr, n)
20:        if swapped = FALSE then
21:            PRINT "Sorted: " ; CALL printArray(arr,n)
22:            PRINT "Passes: ", pass + 1
23:            BREAK
24:        end if
25:        if pass = n - 1 then
26:            PRINT "Passes: ", pass + 1
27:        end if
28:    end for
29: end procedure
```

Algorithm 1.1 Bubble Sort (continued)

```
30: procedure MAIN( )
31:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}, n ← 8
32:     comps ← 0
33:     swaps ← 0
34:     PRINT "Run 1: Original Array", endline
35:     CALL bubbleSort(arr, n, comps, swaps)
36:     PRINT " | Comparisons: ", comps, " | Swaps: ", swaps
37:     PRINT endline
38:     comps ← 0
39:     swaps ← 0
40:     PRINT "Run 2: Already Sorted"
41:     CALL bubbleSort(arr, n, comps, swaps)
42:     PRINT " | Comparisons: ", comps, " | Swaps: ", swaps
43:     PRINT endline
44:     revArr ← {62, 34, 32, 23, 19, 14, 7, 5}
45:     comps ← 0
46:     swaps ← 0
47:     PRINT "Run 3: Reverse Sorted"
48:     CALL bubbleSort(arr, n, comps, swaps)
49:     PRINT " | Comparisons: ", comps, " | Swaps: ", swaps
50:     return 0
51: end procedure
```

1.5 Hand Trace

Pass 1 Full Trace (comparision by comparision)

j	Compare	Swap?	Array after step
0	$34 > 7$	Yes	7 34 23 32 5 62 14 19
1	$34 > 23$	Yes	7 23 34 32 5 62 14 19
2	$34 > 32$	Yes	7 23 32 34 5 62 14 19
3	$34 > 5$	Yes	7 23 32 5 34 62 14 19
4	$34 > 62$	No	7 23 32 5 34 62 14 19
5	$62 > 14$	Yes	7 23 32 5 34 14 62 19
6	$62 > 19$	Yes	7 23 32 5 34 14 19 62

Summary of All Passes

j	Comparisons	Swaps	Array after pass	Notes
1	$7 > 7$	6	7 23 32 5 34 14 19 **62**	62 placed
2	$6 > 23$	3	7 23 5 32 14 19 **34 62**	34 placed
3	$5 > 5$	3	7 5 23 14 19 **32 34 62**	32 placed
4	$4 > 62$	3	5 7 14 19 **23 32 34 62**	array is sorted
5	$3 > 14$	0	5 7 14 19 23 32 34 62	no swaps, so stop
Total	**25**	**15**		

Even though the array is sorted after pass 4, the algorithm looks for a pass with no swaps to stop and therefore needs to run pass 5. Without the `swapped` flag, the algorithm would run 7 passes and 7+6+5+4+3+2+1 = 28 comparisons. The `swapped` flag saves 2 passes and 3 comparisons.

Self-check: Each swap corresponds with one inversion. The original array has 15 inversions: 34 is before 6 smaller values, 7 before 1, 23 before 3, 32 before 2, and 62 before 2. The program should report 15 swaps.

1.6 Correctness

Invariant: after pass k, the last k elements are the k largest values in order. Each pass moves the largest value if the unsorted section to the end since each comparison keeps the larger value moving right.

Termination: after at most n - 1 passes, the last n - 1 elements are final and therefore the first one is as well. When the program stops earlier, it was able to find every neighbor pair in order during the last pass which indicates the array is sorted.

Duplicates: Bubble Sort is **stable** because the test is strict (`arr[j] > arr[j+1]`) which means that equal values are never swapped.

n = 0 or n = 1: the outer loop does not run. Nothing is compared and the array is not changed.

1.7 Complexity Analysis

Case	Input	Passes	Comparisons	Swaps	Time
Best	already sorted	1	n - 1	0	O(n)
Average	random order	about n	about n(n -1)/2	about n(n-1)/4	O(n^2)
Worst	reversed	n - 1	n(n-1)/2	n(n-1)/2	O(n^2)

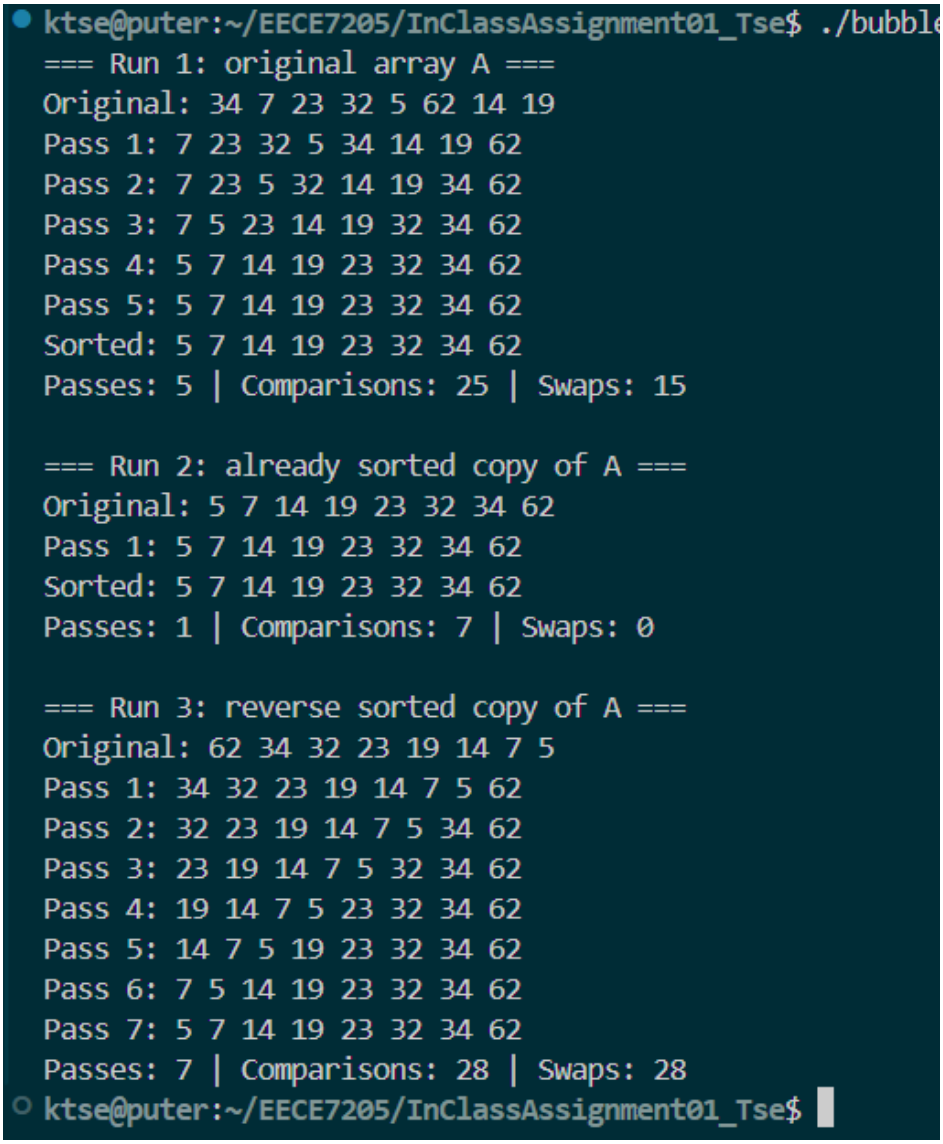
1.8 C++ Program

File Name: `InClassAssignment01\_Problem01\_Tse.cpp`

Compile Instruction:

```
g++ -std=c++17 -Wall -o bubble InClassAssignment01_Problem01_Tse.cpp
```

1.9 Execution Snapshot



1.10 Question 5

Input	Passes	Comparisons	Swaps
Original A: 34 7 23 32 5 62 14 19	5	25	15
Sorted copy: 5 7 14 19 23 32 34 62	1	7	0

Result: when the array is already sorted, the program only makes 1 pass, 7 comparisons,

and 0 swaps.
Explanation: during the first pass of the second run, all neighbors are in the proper order and therefore `arr[j] > arr[j+1]` is always false. No swaps ever occur and the `swapped` remains false. Since no swaps were made, the program exits. A minimum of one pass occurs to verify that the array is sorted. This pass costs n-1 comparisons which means the best case is O(n). If there was not a `swapped` flag, the program would make all 7 passes and 28 comparisons and the best case would be O(n^2). The flag helps improve Bubble Sorts efficiency.

2 Problem 2: Insertion Sort

2.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Insertion Sort algorithm.
- **Input:** an integer array and the size of said array n.
- **Output:** during the algorithm, print the key and the array after the key has been inserted for every pass; after the algorithm, print the sorted array and the total number of comparisons and shifts
- **Constraints:** no built-in sort used; two functions - `printArray` and `insertSort`; all array printing utilizing the corresponding functions; sort occurs in place
- **Edge Cases:** pre-sorted array (best case), reverse sorted array (worst case), duplicate values, array of size 0 or 1

2.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `insertSort(int arr[], int n)`	Section 4, 8
2	Print the key and call `printArray` every pass	See outer loop of `insertSort` for implementation, see Section 9 for output
3	Count comparisons and shifts and print them	See code and output in Section 9
4	Determine what order causes the most and least amount of swaps, run both, and report on the counts	See Section 9 for output and Section 10 for explanation

2.3 Solution Approach

Insertion sort utilizes a key value for each pass. This key is then placed into the correct position by swapping it with the already sorted values that are larger to the left. The algorithm moves from left to right putting each key into the proper position meaning the left most values from the key are sorted.

Design Decisions:

- Each pass only needs to look at the sorted portion of the array (0 to i - 1) [ex. pass 1 -> 1 comps at most, pass 2 -> 2 comps at most, etc.]
- The `key` value is set to i for every pass and sits to the far right of the already sorted values. It is then moved into the correct position based on what is already sorted. Once a value that is less than the `key` is found, comparisons can stop.

2.4 Pseudocode

Algorithm 2.1 Insertion Sort

```
1: procedure printArray(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[k] followed by a space
4:     end for
5:     PRINT newline
6: end procedure
7: procedure insertSort(arr, n, comps, shifts)
8:     PRINT "Original: " ; CALL printArray(arr, n)
9:     for pass ← 1 TO n − 1 do
10:        key ← arr[i]
11:        j = i − 1
12:        while j ≥ 0 AND arr[j] > key do
13:            arr[j + 1] ← arr[j]
14:            j ← j − 1
15:            comps ← comps + 1
16:            shifts ← shifts + 1
17:        end while
18:        if NOT j < 0 then
19:            comps ← comps + 1
20:        end if
21:        arr[j + 1] ← key PRINT "i = ", i, ", key = ", key, ": "; CALL printArray(arr, n)
22:    end for
23:    PRINT "Sorted: "; CALL printArray(arr, n)
24: end procedure
```

Algorithm 2.1 Insertion Sort (continued)

```
25: procedure MAIN( )
26:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}, n ← 8
27:     comps ← 0, shifts ← 0
28:     PRINT "Run 1: Original Array", endline
29:     CALL insertSort(arr, n, comps, shifts)
30:     PRINT "Comparisons: ", comps, " | Shifts: ", shifts, endline
31:     comps ← 0, shifts ← 0
32:     PRINT "Run 2: Fewest Shifts", endline
33:     CALL insertSort(arr, n, comps, shifts)
34:     PRINT "Comparisons: ", comps, " | Shifts: ", shifts, endline
35:     comps ← 0, shifts ← 0
36:     PRINT "Run 3: Most Shifts", endline
37:     CALL insertSort(arr, n, comps, shifts)
38:     PRINT "Comparisons: ", comps, " | Shifts: ", shifts
39:     return 0
40: end procedure
```

2.5 Hand Trace

Full Trace

i	key	Shifts	Array after step
1	7	1 (34)	7 34 23 32 5 62 14 19
2	23	1 (34)	7 23 34 32 5 62 14 19
3	32	1 (34)	7 23 32 34 5 62 14 19
4	5	4 (34, 32, 23, 7)	5 7 23 32 34 62 14 19
5	62	0	5 7 23 32 34 62 14 19
6	14	4 (62, 34, 32,23)	5 7 14 23 32 34 62 19
7	19	4 (62, 34, 32, 23)	5 7 14 19 23 32 34 62
Totals	7 passes	15 shifts	

For each pass, the key is placed into the correction position based on the values that have already been keys. This is done by swapping the key to left if the left value is larger. Once the key is larger than it's neighbor to the left, that pass ends and no more swaps occur (see i = 5 where 0 swaps are made).

Self-check: Each shift corresponds with one inversion. The original array has 15 inversions: 34 is before 6 smaller values, 7 before 1, 23 before 3, 32 before 2, and 62 before 2. The program should report 15 shifts.

2.6 Correctness

Invariant: after pass k, first k + 1 values (index 0 - k) should be sorted in relation to each other. Each pass moves the key value to it's proper place among the already sorted values by comparing it to the values to the left.

Termination: after n - 1 passes, all elements have been compared and shifted to the correct position. There is no early termination.

Duplicates: Insertion Sort is **stable** because the test is strict (`arr[j] > key`) which means that equal values are never swapped.

n = 0 or n = 1: the outer loop does not run. Nothing is compared and the array is not changed.

2.7 Complexity Analysis

Case	Input	Passes	Comparisons	Shifts	Time
Best	already sorted	n - 1	n - 1	0	O(n)
Average	random order	n - 1	about n(n -1)/2	about n(n-1)/4	O(n^2)
Worst	reversed	n - 1	n(n-1)/2	n(n-1)/2	O(n^2)

2.8 C++ Program

File Name: `InClassAssignment01\_Problem02\_Tse.cpp`

Compile Instruction:

```
g++ -std=c++17 -Wall -o insert InClassAssignment01_Problem02_Tse.cpp
```

2.9 Execution Snapshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./insert
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 20 | Shifts: 15

=== Run 2: fewest shifts ===
Original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7 | Shifts: 0

=== Run 3: most shifts ===
Original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 | Shifts: 28
```

2.10 Question 4

Input	Comparisons	Swaps
Fewest Shifts: 5 7 14 19 23 32 34 6	7	0
Most Shifts: 62 34 32 23 19 14 7 5	28	28

Result: The fewest shifts occur when the array is already sorted (best case). The most shifts occur when the array is in reverse order or highest to lowest (worse case).
Explanation: when the algorithm runs on an already sorted array, each pass performs 0 shifts because all values to the left are lower than their neighboring values to the right. This also means that the inner loop is never run. When the algorithm runs on a reverse order

array, each key has to be shifted all the way to the front of the array. This means that every value is moved the maximum number of shifts it could possible be moved $(n(n-1)/2)$.

3 Problem 3: Selection Sort

3.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Selection Sort algorithm.
- **Input:** an integer array and the size of said array n.
- **Output:** during the algorithm print the index, minimum element, and current array for every pass; after the algorithm, print the sorted array and the total number of comparisons and swaps
- **Constraints:** no built-in sort used; two functions - `printArray` and `selectSort`; all array printing utilizing the corresponding functions; sort occurs in place
- **Edge Cases:** pre-sorted array (best case), reverse sorted array (worst case), duplicate values, array of size 0 or 1

3.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `selectSort(int arr[], int n)`	Section 4, 8
2	Print the index and min value, and call `printArray` every pass	See code for implementation, see Section 9 for output
3	Swaps should only occur when the minimum element is in the wrong place (minIdx $\neq$ i)	See code (section 8)
4	Count comparisons and swaps and print them	See code and output in Section 9
5	Compare the number of swaps with Bubble Sort and evaluate the differences	See Section 9 for output, Problem 1 for Bubble Sort, and Section 10 for explanation

3.3 Solution Approach

Selection sort searches the array for the minimum value of the unsorted section and swaps it to the front of the unsorted section/end of the sorted section. The algorithm moves left to right bringing the next smallest element to the current index. This creates a sorted section

to the left of the current index.
Design Decisions:

- Each pass only needs to look at the unsorted portion of the array (n-2-i) [ex. pass 1 -> 7 comps, pass 2 -> 6 comps, etc.]

- The `minIdx` variable stores the current minimum value. For each pass, the values in the unsorted section of the array are compared and `minIdx` is updated accordingly. Once the comparisons are done, the current index and the minimum value are swapped.

3.4 Pseudocode

Algorithm 3.1 Selection Sort

```
1: procedure printArray(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[k] followed by a space
4:     end for
5:     PRINT newline
6: end procedure
7: procedure selectSort(arr, n, comps, swaps)
8:     PRINT "Original: " ; CALL printArray(arr, n)
9:     for pass ← 0 TO n − 2 do
10:        minIdx ← pass
11:        for j ← i + 1 TO n − 1 do
12:            if arr[j] < arr[minIdx] then
13:                minIdx ← j
14:            end if
15:            comps ← comps + 1
16:        end for
17:        if minIdx ≠ i then
18:            SWAP(arr[i], arr[minIdx])
19:            swaps ← swaps + 1
20:        end if
21:        PRINT "Pass ", pass, ": min = ", arr[i], " at index ", minIdx, " -> "
22:        CALL printArray(arr, n)
23:    end for
24:    PRINT "Sorted: " ; CALL printArray(arr, n)
25: end procedure
```

Algorithm 3.1 Selection Sort (continued)

```
26: procedure MAIN( )
27:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}
28:     n ← 8
29:     comps ← 0
30:     swaps ← 0
31:     PRINT "Run 1: Original Array", endline
32:     CALL selectSort(arr, n, comps, swaps)
33:     PRINT "Comparisons: ", comps, " | Swaps: ", swaps
34:     comps ← 0
35:     swaps ← 0
36:     PRINT "Run 2: Already Sorted Array A", endline
37:     CALL selectSort(arr, n, comps, swaps)
38:     PRINT "Comparisons: ", comps, " | Swaps: ", swaps, endline
39:     PRINT endline
40:     comps ← 0
41:     swaps ← 0
42:     revArr ← {62, 34, 32, 23, 19, 14, 7, 5}
43:     PRINT "Run 3: Reverse Sorted Array A", endline
44:     CALL selectSort(revArr, n, comps, swaps)
45:     PRINT "Comparisons: ", comps, " | Swaps: ", swaps, endline
46:     return 0
47: end procedure
```

3.5 Hand Trace

Pass 1 Full Trace (comparision by comparision)

pass	i	Min Index	Min Value	Swapped	Array after step
1	0	4	5	arr[4] -> arr[0]	**5** 7 23 32 34 62 14 19
2	1	1	7	no swap	**5 7** 23 32 34 62 14 19
3	2	6	14	arr[6] -> arr[2]	**5 7 14** 32 34 62 23 19
4	3	7	19	arr[7] -> arr[3]	**5 7 14 19** 34 62 23 32
5	4	6	23	arr[6] -> arr[4]	**5 7 14 19 23** 62 34 32
6	5	7	32	arr[7] -> arr[5]	**5 7 14 19 23 32** 34 62
7	6	6	34	no swap	**5 7 14 19 23 32 34 62**

For each pass, the minimum value is identified and swapped to the current index. The array only needs to run n-1 times because the largest value will automatically end up at the end of

the array when the index n-1 is swapped if needed.
Self-check: Each swap corresponds with one inversion. The original array has 15 inversions: 34 is before 6 smaller values, 7 before 1, 23 before 3, 32 before 2, and 62 before 2. If you calculate the difference between the indexes for each swap, you get 15, the exact number of inversions. [(4-0) + (6 - 2) + (7 - 3) + (6 - 4) + (7 - 5) = 15]

3.6 Correctness

Invariant: after pass k, the first k elements are the k smallest values in order. Each pass moves the smallest value if the unsorted section to the end of the sorted section since the swap moves the smallest value to the left.
Termination: after n - 1 passes, the first n - 1 elements are final and therefore the last one is as well.
Duplicates: Selection Sort is **not stable** because the minimum is swapped with no attempt to preserve the original order (it swaps without assessing what is being put into it's place).
n = 0 or n = 1: the outer loop does not run. Nothing is compared and the array is not changed.

3.7 Complexity Analysis

Case	Input	Passes	Comparisons	Swaps	Time
Best	already sorted	n - 1	n(n-1)/2	0	O(n^2)
Average	random order	n - 1	n(n -1)/2	about (n-1)/2	O(n^2)
Worst	reversed	n - 1	n(n-1)/2	(n-1)	O(n^2)

3.8 C++ Program

File Name: `InClassAssignment01\_Problem03\_Tse.cpp`
Compile Instruction:

```
g++ -std=c++17 -Wall -o select InClassAssignment01_Problem03_Tse.cpp
```

3.9 Execution Snapshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./select
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 | Swaps: 5

=== Run 2: already sorted array A ===
Original: 5 7 14 19 23 32 34 62
Pass 1: min = 5 at index 0 -> 5 7 14 19 23 32 34 62
Pass 2: min = 7 at index 1 -> 5 7 14 19 23 32 34 62
Pass 3: min = 14 at index 2 -> 5 7 14 19 23 32 34 62
Pass 4: min = 19 at index 3 -> 5 7 14 19 23 32 34 62
Pass 5: min = 23 at index 4 -> 5 7 14 19 23 32 34 62
Pass 6: min = 32 at index 5 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 | Swaps: 0

=== Run 3: reverse sorted array A ===
Original: 62 34 32 23 19 14 7 5
Pass 1: min = 5 at index 7 -> 5 34 32 23 19 14 7 62
Pass 2: min = 7 at index 6 -> 5 7 32 23 19 14 34 62
Pass 3: min = 14 at index 5 -> 5 7 14 23 19 32 34 62
Pass 4: min = 19 at index 4 -> 5 7 14 19 23 32 34 62
Pass 5: min = 23 at index 4 -> 5 7 14 19 23 32 34 62
Pass 6: min = 32 at index 5 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 | Swaps: 4
ktse@puter:~/EECE7205/InClassAssignment01_Tse$
```

3.10 Question 5

Algorithm	Passes	Comparisons	Swaps
Selection Sort	7	28	5
Bubble Sort	5	25	15

Result: between bubble sort and selection sort, selection sort makes fewer swaps.
Explanation: during selection sort, a maximum of one swap is made during every pass. Instead of swapping until a certain condition is no longer true as bubble sort does, selection sort assesses the entire array and makes a single swap accordingly. This can be seen in the numbers above with selection sort making only 5 swaps while bubble sort makes 15 swaps.

4 Problem 4: Quick Sort

4.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Selection Sort algorithm.
- **Input:** an integer array and the size of said array n.
- **Output:** during the algorithm print the pivot and array for every partition with each sub-array indented; for the first partition, print i and j; after the algorithm, print the sorted array and the total number of comparisons
- **Constraints:** no built-in sort used; two functions - `printArray` and `selectSort`; all array printing utilizing the corresponding functions; sort occurs in place
- **Edge Cases:** pre-sorted array (worst case), centered pivot (best case), duplicate values, array of size 0 or 1

4.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `quickSort(int arr[], int low, int high)`	Section 4, 8
2	For the 1st partition print i and j, call `printArray` every swap, and print the position of the pivot	See code for implementation, see Section 9 for output
3	For every recursive call, print the pivot and the sub-array by calling printArray(arr + low, high - low + 1) with each line indented by recursion depth	See code for implementation and Section 9 for output
4	Count comparisons and print them	See code and output in Section 9
5	Run on a sorted array, count comparisons, explain why this is worst case for last element as pivot, and suggest a way to avoid it	See Section 9 for output, and Section 10 for explanation

4.3 Solution Approach

Quick sort selects a pivot value to use as a center value and positions all other elements in the array around the pivot. If a value is less than the pivot it is moved to the left. If a value is greater than the pivot it is moved to the right. This technique is then recursively applied to the values on either side of the pivot until the sub-arrays are only a single value.
Design Decisions:

- While there are few methods to select a pivot, this example will utilize the last value pivot method. This means that the value at the end of the array will be designated as the pivot.

- To help with the ordering and separating of the array, a `partition` helper function will handle swapping elements and returning the location of the pivot.

4.4 Pseudocode

Algorithm 4.1 Quick Sort

```
1: procedure PRINTARRAY(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[i] followed by a space
4:     end for
5:     PRINT newline
6: end procedure
```

Algorithm 4.1 Quick Sort (continued)

```
7: procedure PARTITION(arr, low, high, pass, comps)
8:     pivot ← arr[high]
9:     PRINT tab (pass amount of times), "pivot = ", pivot, ": "
10:    CALL printArray(arr, low, high)
11:    i ← low − 1
12:    j ← low
13:    for j ← low TO high do
14:        if arr[j] ≤ pivot then
15:            i ← i + 1
16:            SWAP(arr[i], arr[j])
17:            if i ≠ j then
18:                if (high − low) + 1 = 8 then
19:                    PRINT "i = ", i, ", j = ", j, ", "
20:                end if
21:                PRINT tab (pass amount of times), "pivot = ", pivot, ": "
22:                CALL printArray(arr, low, high)
23:            end if
24:        end if
25:        comps ← comps + 1
26:    end for
27:    SWAP(arr[i + 1], arr[high])
28:    if (i + 1) ≠ high then
29:        if (high − low) + 1 = 8 then
30:            PRINT "i = ", i, ", j = ", j, ", "
31:        end if
32:        PRINT tab (pass amount of times), "pivot = ", pivot, ": "
33:        CALL printArray(arr, low, high)
34:    end if
35:    return i + 1;
36: end procedure
37: procedure QUICKSORT(arr, low, high, pass, comps)
38:    if low < high then
39:        p ← partition(arr, low, high, pass, comps)
40:        pass ← pass + 1
41:        quickSort(arr, low, p - 1, pass, comps)
42:        quickSort(arr, p + 1, high, pass, comps)
43:    end if
44: end procedure
```

Algorithm 4.1 Quick Sort (continued)

```
45: procedure MAIN( )
46:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}
47:     n ← 8
48:     pass ← 0
49:     comps ← 0
50:     PRINT "Run 1: Original Array", endline
51:     PRINT "Original: "; CALL printArray(arr, n)
52:     CALL quickSort(arr, 0, 7, pass, comps)
53:     PRINT "Sorted: "; CALL printArray(arr, n)
54:     PRINT "Comparisons: ", comps, endline
55:     PRINT endline
56:     PRINT "Run 2: Already Sorted", endline
57:     pass ← 0
58:     comps ← 0
59:     PRINT "Original: "; CALL printArray(arr, n)
60:     CALL quickSort(arr, 0, 7, pass, comps)
61:     PRINT "Sorted: "; CALL printArray(arr, n)
62:     PRINT "Comparisons: ", comps, endline
63:     PRINT endline
64:     PRINT "Run 3: Reverse Sorted Copy of A", endline
65:     revArr ← {62, 34, 32, 23, 19, 14, 7, 5}
66:     pass ← 0
67:     comps ← 0
68:     PRINT "Original: "; CALL printArray(revArr, n)
69:     CALL quickSort(revArr, 0, 7, pass, comps)
70:     PRINT "Sorted: "; CALL printArray(revArr, n)
71:     PRINT "Comparisons: ", comps, endline
72:     return 0
73: end procedure
```

4.5 Hand Trace

Partition Loop (j = low to high - 1)

j	i	Compare	Swap?	Array after step
0	-1	34 <= 19	No	34 7 23 32 5 62 14 19
1	0	7 <= 19	Yes	**7 34** 23 32 5 62 14 19
2	0	23 <= 19	No	7 34 23 32 5 62 14 19
3	0	32 <= 19	No	7 34 23 32 5 62 14 19
4	1	5 <= 19	Yes	7 **5** 23 32 **3**4 62 14 19
5	1	62 <= 19	No	7 5 23 32 34 62 14 19
6	2	14 <= 19	Yes	7 5 **14** 32 34 62 **23** 19
7	2	Swap Pivot	Yes	7 5 14 **19** 34 62 23 **32**

Summary of All Passes

Low	High	Pivot	Array after partition	Swaps	Notes
0	2	14	7 5 14	0	left of original pivot
4	7	32	23 32 34 62	2	right of original pivot
0	1	5	5 7	1	left of left array
6	7	62	34 62	0	right of right array

After each pass, the array is sectioned into smaller sub-arrays until the sub-array is one value (not shown in chart). Each sub array is treated the same as the first original partition.
Self-check: Each sub-array should result in a sorted array on the corresponding side of the previous pivot. If you rebuild the array from the smallest arrays up through the pivots you get ([5 7] + 14 + 19 + [23] + 32 + [34 62]) which is the sorted array.

4.6 Correctness

Invariant: when arr[last] is chosen as the pivot, and you track indices i and j, the array is partitioned into four sections ($\leq$ pivot, $>$ pivot, unprocessed, the pivot) which are continually split until the sub-array is of length 1.
Termination: when the length of a given sub-array is 1, the sub-array is sorted and no more partitions are created.
Duplicates: Quick Sort is **not stable** because values are swapped with no attempt to preserve the original order (it swaps without assessing what is being put into it's place).

n = 0 or n = 1: the outer loop does not run. Nothing is compared and the array is not changed.

4.7 Complexity Analysis

Case	Input	Passes	Comparisons	Time
Best	perfectly split	n	nlogn	O($nlogn$)
Average	random order	n	about nlogn	O($nlogn$)
Worst	already sorted	n	n(n-1)/2	O(n^2)

4.8 C++ Program

File Name: `InClassAssignment01\_Problem04\_Tse.cpp`

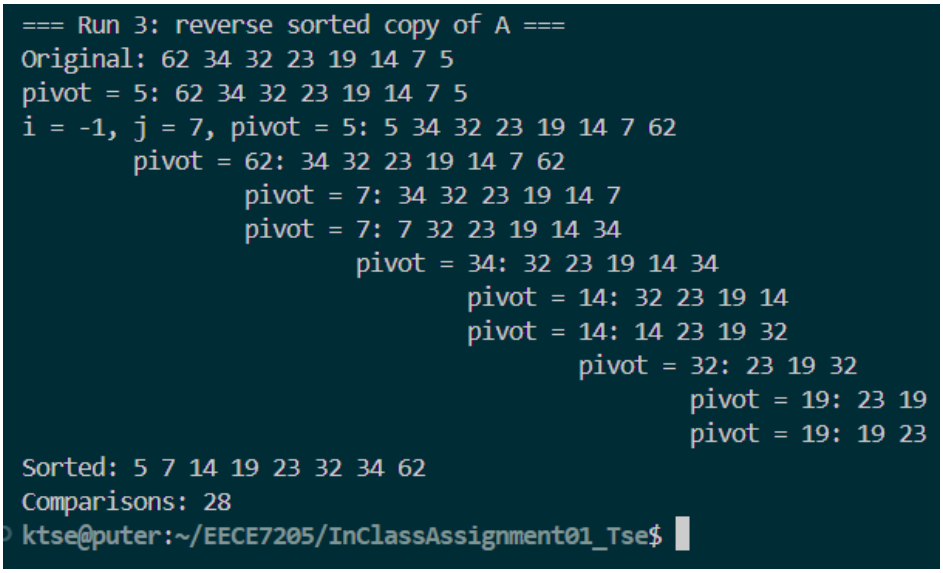
Compile Instruction:

```
g++ -std=c++17 -Wall -o quick InClassAssignment01_Problem04_Tse.cpp
```

4.9 Execution Snapshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./quick
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
pivot = 19: 34 7 23 32 5 62 14 19
i = 0, j = 1, pivot = 19: 7 34 23 32 5 62 14 19
i = 1, j = 4, pivot = 19: 7 5 23 32 34 62 14 19
i = 2, j = 6, pivot = 19: 7 5 14 32 34 62 23 19
i = 2, j = 7, pivot = 19: 7 5 14 19 34 62 23 32
        pivot = 14: 7 5 14
                pivot = 5: 7 5
                pivot = 5: 5 7
        pivot = 32: 34 62 23 32
        pivot = 32: 23 62 34 32
        pivot = 32: 23 32 34 62
                pivot = 62: 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14

=== Run 2: already sorted copy of A ===
Original: 5 7 14 19 23 32 34 62
pivot = 62: 5 7 14 19 23 32 34 62
        pivot = 34: 5 7 14 19 23 32 34
                pivot = 32: 5 7 14 19 23 32
                        pivot = 23: 5 7 14 19 23
                                pivot = 19: 5 7 14 19
                                        pivot = 14: 5 7 14
                                                pivot = 7: 5 7
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28
ktse@puter:~/EECE7205/InClassAssignment01_Tse$
```



4.10 Question 5

Input	Comparisons	Case
34 7 23 32 5 62 14 19	14	Average
5 7 14 19 23 32 34 62	28	Worse

Result: worse case (sorted array) results in n(n-1)/2 comparisons when the pivot is the last element of the array
Explanation: when the last element of the array is the pivot for an already sorted array, all elements are considered smaller than the pivot and no swaps are made and the partition simply removes the pivot. This occurs for all partitions which means the algorithm looks at n + (n - 1) + (n - 2) ... sized sub-arrays.
Suggested Fix: one way to avoid this issue would be to select a different pivot. Utilizing either a middle or random pivot would prevent this issue from occurring because there would actually be a split of values around the pivot.

5 Problem 5: Merge Sort

5.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Selection Sort algorithm.
- **Input:** an integer array and the size of said array n.
- **Output:** during the algorithm print each split sub-array and each merged sub-array; after the algorithm, print the sorted array and the total number of comparisons, and the number of comparisons in the final merge

- **Constraints:** no built-in sort used; two functions - `printArray` and `mergeSort`; all array printing utilizing the corresponding functions; sort occurs in place

- **Edge Cases:** pre-sorted array (best case), alternating elements (worst case), duplicate values, array of size 0 or 1

5.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `mergeSort(int arr[], int low, int high)`	Section 4, 8
2	Every time a sub-array is split, call `printArray`	See code for implementation, see Section 9 for output
3	Every time sub-arrays are merged, call `printArray`	See code for implementation and Section 9 for output
4	Count comparisons and print them and the amount made in the final merge	See code and output in Section 9
5	Run on a sorted and reversed array, explain why the number of comparisons doesn't change as much as other sorting algorithms, and state the memory needs of merge sort	See Section 9 for output, and Section 10 for explanation

5.3 Solution Approach

Merge sort continues to divide the original arrays in half into sub-arrays until each sub-array contains only one element. It then sorts and merges the halves back together keeping each element in sorted order as the sub-arrays are merged together.
Design Decisions:

- In order to split the arrays, the mid-point/split index will be calculated by (high + low)/2.

- To help with the ordering and merging of the array, a `merge` helper function will handle combining sub-arrays back together in sorted order.

5.4 Pseudocode

Algorithm 5.1 Merge Sort

```
1: procedure PRINTARRAY(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[i] followed by a space
4:     end for
5:     PRINT newline
6: end procedure
7: procedure MERGE(arr, low, mid, high, comps, final)
8:     L ← arr[low...mid]
9:     R ← arr[mid + 1...high]
10:    i ← 0, j ← 0, k ← low
11:    while i < SIZEOF(L) AND j < SIZEOF(R) do
12:        if L[i] ≤ R[j] then
13:            arr[k] ← L[i]
14:            i ← i + 1
15:        else
16:            arr[k] ← R[j]
17:            j ← j + 1
18:        end if
19:        k ← k + 1
20:        comps ← comps + 1
21:        if (high − low + 1) = 8 then
22:            final ← final + 1
23:        end if
24:    end while
25:    while i < SIZEOF(L) do
26:        arr[k] ← L[i]
27:        i ← i + 1
28:        k ← k + 1
29:    end while
30:    while j < SIZEOF(R) do
31:        arr[k] ← R[j]
32:        j ← j + 1
33:        k ← k + 1
34:    end while
35:    PRINT "merged: "; CALL printArray(arr + low, high - low + 1)
36: end procedure
37: procedure MERGESORT(arr, low, high, comps, final)
38:    if low < high then
39:        mid ← (low + high)/2
40:        PRINT "split: "; CALL printArray(arr + low, mid - low + 1)
41:        mergeSort(arr, low, mid, comps, final)
42:        PRINT "split: "; CALL printArray(arr + mid + 1, high - mid)
43:        mergeSort(arr, mid + 1, high, comps, final)
44:        merge(arr, low, mid, high, comps, final)
45:    end if
46: end procedure
```

Algorithm 5.1 Merge Sort (continued)

```
47: procedure MAIN( )
48:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}
49:     n ← SIZEOF(arr)
50:     comps ← 0
51:     finalComps ← 0
52:     PRINT "Run 1: Original Array", endline
53:     PRINT "Original: "; CALL printArray(arr, n)
54:     CALL mergeSort(arr, 0, n - 1, comps, finalComps)
55:     PRINT "Sorted: "; CALL printArray(arr, n)
56:     PRINT "Comparisons: ", comps, " (final merge: ", finalComps, ")", endline
57:     PRINT endline
58:     comps ← 0
59:     finalComps ← 0
60:     PRINT "Run 2: Already Sorted Copy of A", endline
61:     PRINT "Original: "; CALL printArray(arr, n)
62:     CALL mergeSort(arr, 0, n - 1, comps, finalComps)
63:     PRINT "Sorted: "; CALL printArray(arr, n)
64:     PRINT "Comparisons: ", comps, " (final merge: ", finalComps, ")", endline
65:     PRINT endline
66:     comps ← 0
67:     finalComps ← 0
68:     PRINT "Run 3: Reverse Sorted Array A", endline
69:     reversed ← {62, 34, 32, 23, 19, 14, 7, 5}
70:     PRINT "Original: "; CALL printArray(reversed, n)
71:     CALL mergeSort(reversed, 0, n - 1, comps, finalComps)
72:     PRINT "Sorted: "; CALL printArray(reversed, n)
73:     PRINT "Comparisons: ", comps, " (final merge: ", finalComps, ")", endline
74:     return 0
75: end procedure
```

5.5 Hand Trace

Splits Trace

Original	Left	Right	Another Split?
34, 7, 23, 32, 5, 62, 14, 19	34, 7, 23, 32	5, 62, 14, 19	Yes
34, 7, 23, 32	34, 7	23, 32	Yes
5, 62, 14, 19	5, 62	14, 19	Yes
34, 7	34	7	No
23, 32	23	32	No
5, 62	5	62	No
14, 19	14	19	No

Merge Trace

Left	Right	Merged	Done?
14	19	14, 19	No
5	62	5, 62	No
23	32	23, 32	No
34	7	7, 34	No
5, 62	14, 19	5, 14, 19, 62	No
7, 34	23, 32	7, 23, 32, 34	No
7, 23, 32, 34	5, 14, 19, 62	5, 7, 14, 19, 23, 32, 34, 62	Yes

After each split, the array is sectioned into smaller sub-arrays until the sub-array is one value. Each sub array is then merged back together in sorted order.
Self-check: Each merged sub-arrays should result in a sorted array. The merged column of the Merge Trace shows that each array is sorted as a result of the merge.

5.6 Correctness
Invariant: at the start of pass p, the sub-array A[p...k-1] holds k-p smallest elements of the left and right sub-arrays sorted in order.
Termination: when the length of a given sub-array is 1, the sub-arrays are no longer split and the program moves to merge them. The sub-arrays are merged until you have recreated the entire original array.
Duplicates: Merge Sort is **stable** because the merge function utilizes $\leq$ when combining the left and right arrays meaning when two elements are the same, the left one is chosen. Since the left and right sub-arrays are created based on the original array, the intial order is

preserved.
n = 0 or n = 1: the outer loop does not run. Nothing is compared and the array is not changed.

5.7 Complexity Analysis

Case	Input	Comparisons	Time
Best	already sorted	about 0.5nlogn	O($nlogn$)
Average	random order	about nlogn - 1.26n	O($nlogn$)
Worst	alternating values	about nlogn - n	O($nlogn$)

5.8 C++ Program

File Name: `InClassAssignment01\_Problem05\_Tse.cpp`
Compile Instruction:

```
g++ -std=c++17 -Wall -o merge InClassAssignment01_Problem05_Tse.cpp
```

5.9 Execution Snapshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./merge
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
split: 34
split: 7
merged: 7 34
split: 23 32
split: 23
split: 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
split: 5
split: 62
merged: 5 62
split: 14 19
split: 14
split: 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)
```

```
=== Run 2: already sorted copy of A ===
Original: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
split: 5
split: 7
merged: 5 7
split: 14 19
split: 14
split: 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
split: 23
split: 32
merged: 23 32
split: 34 62
split: 34
split: 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)
```

```
=== Run 3: reverse sorted array A ===
Original: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
split: 62
split: 34
merged: 34 62
split: 32 23
split: 32
split: 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
split: 19
split: 14
merged: 14 19
split: 7 5
split: 7
split: 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)
ktse@puter:~/EECE7205/InClassAssignment01_Tse$
```

5.10 Question 5

Input	Comparisons	Final Comparisons
Sorted Array	12	4
Reverse Array	12	4

Result: the number of comparisons is exactly the same for a sorted and reversed array
Explanation: merge sort does not have an early termination condition and therefore will run the same number of times on all arrays of the same size. The comparison numbers are exactly the same because the base logic is running exactly the same way (the only difference is in a reversed array there are more swaps). Meanwhile, Bubble Sort has an exit condition which allows a pre-sorted array to exit very early and avoid a large number of comparisons compared to a reversed array. Quick Sort with the pivot set to the end of the array sees more comparisons especially with a pre-sorted array because the reference value affects the number of comparisons.
Memory Assessment: Merge sort needs more memory than both Bubble Sort and Quick Sort because it requires temp arrays to perform the the rearrangements. An additional space of O(n) is needed to store a copy of the original array as reference.

6 Problem 6: Heap Sort

6.1 Understanding the Algorithm

- **Goal:** sort `A = {34, 7, 23, 32, 5, 62, 14, 19}` by increasing order using the Selection Sort algorithm.

- **Input:** an integer array and the size of said array n.

- **Output:** during the algorithm print max heap once made and print the heap and sorted section of the array after every pass using; after the algorithm, print the sorted array and the total number of swaps made in `siftDown`

- **Constraints:** no built-in sort used; two functions - `printArray` and `heapSort`; all array printing utilizing the corresponding functions; sort occurs in place

- **Edge Cases:** pre-sorted array (worst case), centered pivot (best case), duplicate values, array of size 0 or 1

6.2 Requirement Checklist

#	Requirement	Where satisfied
1	Functions `printArray(const int arr[], int n)` and `heapSort(int arr[], int low, int high)`	Section 4, 8
2	Print the max heap after building by calling `printArray` and draw the heap as a binary tree	See code for implementation, see Section 9 for output and tree
3	For every pass, print the tree using printArray(arr, end) and print the sorted section of the array by calling printArray(arr + end, n - end)	See code for implementation and Section 9 for output
4	Count swaps inside `siftDown` and print them	See code and output in Section 9
5	Explain why Heap Sort is or isn't stable and use a example array that contains two equal values to demonstrate	See Section 9 for output, and Section 10 for explanation

6.3 Solution Approach

Heap sort converts the array into a binary heap and uses that structure to pop values off and sort them.

Design Decisions:

- This algorithm utilizes a max binary heap to build a sorted list from right to left.

- To help with maintaing a binary heap, a `siftDown` helper function will handle re-organizing the heap after an element is removed.

6.4 Pseudocode

Algorithm 6.1 Heap Sort

```
1: procedure printArray(arr, n)
2:     for i ← 0 TO n − 1 do
3:         PRINT arr[i] followed by a space
4:     end for
5: end procedure
6: procedure siftDown(arr, root, size, swaps, comps)
7:     while 2 × root + 1 < size do
8:         child ← 2 × root + 1
9:         if (child + 1 < size) AND (arr[child + 1] > arr[child]) then
10:            child ← child + 1
11:        end if
12:        comps ← comps + 1
13:        if arr[root] ≥ arr[child] then
14:            BREAK
15:        end if
16:        comps ← comps + 1
17:        SWAP(arr[root], arr[child])
18:        root ← child
19:        swaps ← swaps + 1
20:    end while
21: end procedure
22: procedure heapSort(arr, n, swaps, comps)
23:    for i ← (n/2) − 1 DOWNTO 0 do
24:        CALL siftDown(arr, i, n, swaps, comps)
25:    end for
26:    PRINT "Heap: "; CALL printArray(arr, n)
27:    PRINT newline
28:    for end ← n − 1 DOWNTO 1 do
29:        SWAP(arr[0], arr[end])
30:        CALL siftDown(arr, 0, end, swaps, comps)
31:        PRINT "end = ", end, ": heap: "
32:        CALL printArray(arr, end)
33:        PRINT " sorted: "
34:        CALL printArray(arr + end, n - end)
35:        PRINT newline
36:    end for
37: end procedure
```

Algorithm 6.1 Heap Sort (continued)

```
38: procedure MAIN( )
39:     arr ← {34, 7, 23, 32, 5, 62, 14, 19}
40:     n ← 8
41:     swaps ← 0
42:     comps ← 0
43:     PRINT "Run 1: Original Array", endline
44:     PRINT "Original: "; CALL printArray(arr, n)
45:     PRINT newline
46:     CALL heapSort(arr, n, swaps, comps)
47:     PRINT "Sorted: "; CALL printArray(arr, n)
48:     PRINT newline
49:     PRINT "Swaps in siftDown: ", swaps, " | Comparisons: ", comps, endline, endline
50:     swaps ← 0
51:     comps ← 0
52:     PRINT "Run 2: Array with Duplicate Value", endline
53:     dupArr ← {34, 7, 23, 34, 5, 62, 14, 19}
54:     PRINT "Original: "; CALL printArray(dupArr, n)
55:     PRINT newline
56:     CALL heapSort(dupArr, n, swaps, comps)
57:     PRINT "Sorted: "; CALL printArray(dupArr, n)
58:     PRINT newline
59:     PRINT "Swaps in siftDown: ", swaps, " | Comparisons: ", comps, endline, endline
60:     swaps ← 0
61:     comps ← 0
62:     PRINT "Run 3: Sorted Array A", endline
63:     PRINT "Original: "; CALL printArray(arr, n)
64:     PRINT newline
65:     CALL heapSort(arr, n, swaps, comps)
66:     PRINT "Sorted: "; CALL printArray(arr, n)
67:     PRINT newline
68:     PRINT "Swaps in siftDown: ", swaps, " | Comparisons: ", comps, endline, endline
69:     revArr ← {62, 34, 32, 23, 19, 14, 7, 5}
70:     swaps ← 0
71:     comps ← 0
72:     PRINT "Run 4: Reverse Array A", endline
73:     PRINT "Original: "; CALL printArray(revArr, n)
74:     PRINT newline
75:     CALL heapSort(revArr, n, swaps, comps)
76:     PRINT "Sorted: "; CALL printArray(revArr, n)
77:     PRINT newline
78:     PRINT "Swaps in siftDown: ", swaps, " | Comparisons: ", comps, endline
79:     return 0
80: end procedure
```

6.5 Hand Trace

Heap Sort

end	heap	sorted array
X	62 32 34 19 5 23 14 7	[ ]
7	34 32 23 19 5 7 14	62
6	32 19 23 14 5 7	34 62
5	23 19 7 14 5	32 34 62
4	19 14 7 5	23 32 34 62
3	14 5 7	19 23 32 34 62
2	7 5	14 19 23 32 34 62
1	5	5 14 19 23 32 34 62

After each pass, the array can be split into the heap and the sorted section of the array. For each pass, the top of the heap is moved from the heap into the sorted section of the array
Self-check: If the heap is maintained properly, the front of the array is always the next element to be placed into sorted order from right to left as seen in the chart above.

6.6 Correctness

Invariant: for each pass, the array is split into the heap from [0 : end] and the sorted array from [end + 1 : n].
Termination: when the heap is of size 1, the array has been sorted and the function exits.
Duplicates: see section 10 for explanation
n = 0 or n = 1: the loops do not run. Nothing is compared and the array is not changed.

6.7 Complexity Analysis

Case	Comparisons	Swaps	Time
Best	nlogn	nlogn	O($nlogn$)
Average	about 2nlogn	about nlogn	O($nlogn$)
Worst	2nlogn	nlogn	O($nlogn$)

6.8 C++ Program

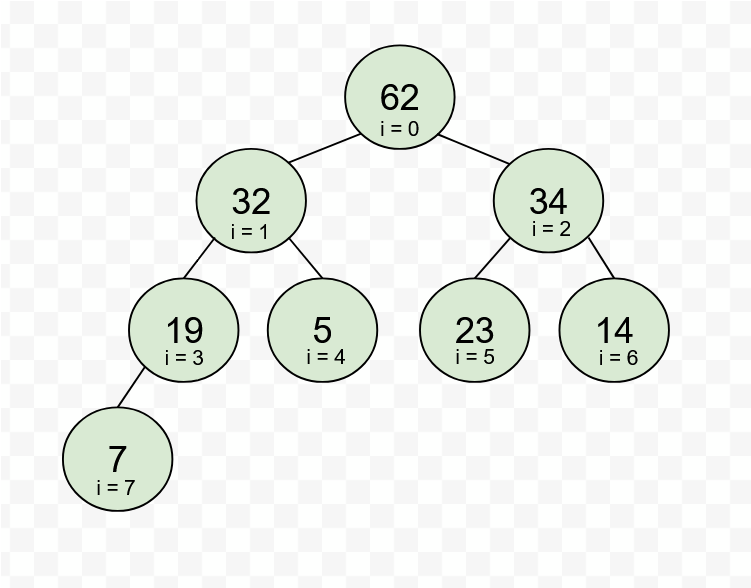
File Name: `InClassAssignment01\_Problem06\_Tse.cpp`
Compile Instruction:

```
g++ -std=c++17 -Wall -o heap InClassAssignment01_Problem06_Tse.cpp
```

6.9 Execution Snapshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./heap
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7:        heap: 34 32 23 19 5 7 14        sorted: 62
end = 6:        heap: 32 19 23 14 5 7   sorted: 34 62
end = 5:        heap: 23 19 7 14 5      sorted: 32 34 62
end = 4:        heap: 19 14 7 5         sorted: 23 32 34 62
end = 3:        heap: 14 5 7    sorted: 19 23 32 34 62
end = 2:        heap: 7 5       sorted: 14 19 23 32 34 62
end = 1:        heap: 5         sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 12 | Comparisons: 27

=== Run 2: array with duplicate value ===
Original: 34 7 23 34 5 62 14 19
Heap: 62 34 34 19 5 23 14 7
end = 7:        heap: 34 19 34 7 5 23 14        sorted: 62
end = 6:        heap: 34 19 23 7 5 14   sorted: 34 62
end = 5:        heap: 23 19 14 7 5      sorted: 34 34 62
end = 4:        heap: 19 7 14 5         sorted: 23 34 34 62
end = 3:        heap: 14 7 5    sorted: 19 23 34 34 62
end = 2:        heap: 7 5       sorted: 14 19 23 34 34 62
end = 1:        heap: 5         sorted: 7 14 19 23 34 34 62
Sorted: 5 7 14 19 23 34 34 62
Swaps in siftDown: 13 | Comparisons: 28
```



```
=== Run 3: sorted array A ===
Original: 5 7 14 19 23 32 34 62
Heap: 62 23 34 19 5 32 14 7
end = 7:        heap: 34 23 32 19 5 7 14        sorted: 62
end = 6:        heap: 32 23 14 19 5 7   sorted: 34 62
end = 5:        heap: 23 19 14 7 5      sorted: 32 34 62
end = 4:        heap: 19 7 14 5         sorted: 23 32 34 62
end = 3:        heap: 14 7 5    sorted: 19 23 32 34 62
end = 2:        heap: 7 5       sorted: 14 19 23 32 34 62
end = 1:        heap: 5         sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 15 | Comparisons: 31

=== Run 4: reverse array A ===
Original: 62 34 32 23 19 14 7 5
Heap: 62 34 32 23 19 14 7 5
end = 7:        heap: 34 23 32 5 19 14 7        sorted: 62
end = 6:        heap: 32 23 14 5 19 7   sorted: 34 62
end = 5:        heap: 23 19 14 5 7      sorted: 32 34 62
end = 4:        heap: 19 7 14 5         sorted: 23 32 34 62
end = 3:        heap: 14 7 5    sorted: 19 23 32 34 62
end = 2:        heap: 7 5       sorted: 14 19 23 32 34 62
end = 1:        heap: 5         sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 9 | Comparisons: 23
ktse@puter:~/EECE7205/InClassAssignment01_Tse$
```

6.10 Question 5

Result: Heap sort is not stable

Explanation: heap sort is not stable because there is no guarantee that the original input order is preserved. If you look at the execution in section 9, you can see two values of 34 in the trace. Even if you assume that during `siftDown`, the original order is maintained, when the values are added to the sorted section of the array, the order will be flipped since the sorted array is built right to left.

7 Problem 7: Comparing the Six Algorithms

7.1 Comparisons Chart

Sort Method	A	A Sorted	A Reversed
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	27	31	23

7.2 Run Time Chart

Sort Method	1,000	5,000	10,000
Bubble Sort	1.0845 ms	25.9838 ms	96.5526 ms
Insertion Sort	0.25616 ms	6.15786 ms	26.34526 ms
Selection Sort	0.753066 ms	18.9379 ms	68.3022 ms
Quick Sort	0.049459 ms	0.279662 ms	0.619167 ms
Merge Sort	0.073752 ms	0.462472 ms	1.10664 ms
Heap Sort	0.064879 ms	0.446183 ms	1.11142 ms

7.3 Execution Screenshot

```
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ g++ -Wall InClassAssignment01_Problem07_Tse.cpp -o prob7
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./prob7
=== Sorting 1000 items ===
Bubble Sort: 1.0845 ms -> 116 311 336 358 359 467 477 490 545 653
Insertion Sort: 0.25616 ms -> 116 311 336 358 359 467 477 490 545 653
Selection Sort: 0.753066 ms -> 116 311 336 358 359 467 477 490 545 653
Quick Sort: 0.049459 ms -> 116 311 336 358 359 467 477 490 545 653
Merge Sort: 0.073752 ms -> 116 311 336 358 359 467 477 490 545 653
Heap Sort: 0.064879 ms -> 116 311 336 358 359 467 477 490 545 653
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ g++ -Wall InClassAssignment01_Problem07_Tse.cpp -o prob7
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./prob7
=== Sorting 5000 items ===
Bubble Sort: 25.9838 ms -> 4 51 58 78 86 86 88 107 195 198
Insertion Sort: 6.15786 ms -> 4 51 58 78 86 86 88 107 195 198
Selection Sort: 18.9379 ms -> 4 51 58 78 86 86 88 107 195 198
Quick Sort: 0.279662 ms -> 4 51 58 78 86 86 88 107 195 198
Merge Sort: 0.462472 ms -> 4 51 58 78 86 86 88 107 195 198
Heap Sort: 0.446183 ms -> 4 51 58 78 86 86 88 107 195 198
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ g++ -Wall InClassAssignment01_Problem07_Tse.cpp -o prob7
ktse@puter:~/EECE7205/InClassAssignment01_Tse$ ./prob7
=== Sorting 10000 items ===
Bubble Sort: 96.5526 ms -> 1 2 4 36 36 75 96 97 107 111
Insertion Sort: 26.3452 ms -> 1 2 4 36 36 75 96 97 107 111
Selection Sort: 68.3022 ms -> 1 2 4 36 36 75 96 97 107 111
Quick Sort: 0.619167 ms -> 1 2 4 36 36 75 96 97 107 111
Merge Sort: 1.10664 ms -> 1 2 4 36 36 75 96 97 107 111
Heap Sort: 1.11142 ms -> 1 2 4 36 36 75 96 97 107 111
ktse@puter:~/EECE7205/InClassAssignment01_Tse$
```

7.4 Written Analysis

- Bubble Sort, Insertion Sort, and Selection Sort all grow like n^2, while Quick Sort, Merge Sort, and Heap Sort grow like $nlogn$.

- The run time chart easily displays the large jump in time complexity for the first three sorting methods compared to the second three.

- I would choose to use Quick Sort for a large random array as it is by far the fastest runtime of all the sorting algorithms as seen in the run time chart.

- I would likely pick Insertion Sort for an almost sorted array because insertion sort only runs the center loop when swaps are needed. If the array is almost sorted, the time complexity gets much closer to O(n).

7.5 Pseudocode

Algorithm 7.1 Sorting Algorithm Timing Comparison

```
1: procedure MAIN( )
2:     SEED random number generator with current time
3:     size ← 1000
4:     generated ← arr[size]
5:     for i ← 0 TO size − 1 do
6:         generated[i] ← RANDOM(1, 100000)
7:     end for
8:     bubble1 ← COPY(generated)
9:     insert1 ← COPY(generated)
10:    select1 ← COPY(generated)
11:    quick1 ← COPY(generated)
12:    merge1 ← COPY(generated)
13:    heap1 ← COPY(generated)
14:    PRINT "Sorting ", size, " items", endline
                                                    ▷ Bubble Sort
15:    start ← CURRENT TIME
16:    CALL bubbleSort(bubble1, size)
17:    end ← CURRENT TIME
18:    duration ← end − start
19:    PRINT "Bubble Sort: ", duration, " ms -> "
20:    CALL printArray(bubble1, 10)
                                                    ▷ Insertion Sort
21:    start ← CURRENT TIME
22:    CALL insertSort(insert1, size)
23:    end ← CURRENT TIME
24:    duration ← end − start
25:    PRINT "Insertion Sort: ", duration, " ms -> "
26:    CALL printArray(insert1, 10)
```

Algorithm 7.1 Heap Sort (continued)

```
                                                                ▷ Selection Sort
    start ← CURRENT TIME
    CALL selectSort(select1, size)
    end ← CURRENT TIME
    duration ← end − start
    PRINT "Selection Sort: ", duration, " ms -> "
    CALL printArray(select1, 10)
                                                                ▷ Quick Sort
    start ← CURRENT TIME
    CALL quickSort(quick1, 0, size - 1)
    end ← CURRENT TIME
    duration ← end − start
    PRINT "Quick Sort: ", duration, " ms -> "
    CALL printArray(quick1, 10)
                                                                ▷ Merge Sort
    start ← CURRENT TIME
    CALL mergeSort(merge1, 0, size - 1)
    end ← CURRENT TIME
    duration ← end − start
    PRINT "Merge Sort: ", duration, " ms -> "
    CALL printArray(merge1, 10)
                                                                ▷ Heap Sort
    start ← CURRENT TIME
    CALL heapSort(heap1, size)
    end ← CURRENT TIME
    duration ← end − start
    PRINT "Heap Sort: ", duration, " ms -> "
    CALL printArray(heap1, 10)
end procedure
```

7.6 C++ Program

File Name: `InClassAssignment01\_Problem07\_Tse.cpp`

Compile Instruction:

```
g++ -std=c++17 -Wall InClassAssignment01_Problem07_Tse.cpp -o prob7
```